

Install Required Libraries

pip install tensorflow matplotlib numpy pillow scikit-learn

Folder Structure

Download **20–30 fashion images** from the web and organize them like this:

fashion_dataset/

```
| — train/
|   |
|   | — t_shirt/
|   |   tshirt1.jpg
|   |   tshirt2.jpg
|   |   ...
|   |
|   | — shoe/
|   |   shoe1.jpg
|   |   ...
|   |
|   | — bag/
|   |   bag1.jpg
|   |   ...
```

```
| — validation/
|   | — t_shirt/
|   | — shoe/
|   | — bag/
```

```
| — test/
|   | — t_shirt/
|   | — shoe/
|   | — bag/
```

Example split:

- **Training:** 18 images
 - **Validation:** 6 images
 - **Testing:** 6–10 images
-

Task 1: Load Dataset

```
import tensorflow as tf

IMG_SIZE = (128,128)

train_dataset = tf.keras.preprocessing.image_dataset_from_directory(

    "fashion_dataset/train",

    image_size=IMG_SIZE,

    batch_size=8

)

validation_dataset = tf.keras.preprocessing.image_dataset_from_directory(

    "fashion_dataset/validation",

    image_size=IMG_SIZE,

    batch_size=8

)

test_dataset = tf.keras.preprocessing.image_dataset_from_directory(

    "fashion_dataset/test",

    image_size=IMG_SIZE,

    batch_size=1,

    shuffle=False

)

class_names = train_dataset.class_names

print(class_names)
```

Example Output

```
['bag', 'shoe', 't_shirt']
```

Task 2: Build a Basic CNN

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
```

```
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Rescaling
```

```
)
```

```
model = Sequential([
```

```
    Rescaling(
        1./255,
        input_shape=(128,128,3)
    ),
```

```
    Conv2D(
        32,
        (3,3),
        activation="relu"
    ),
```

```
    MaxPooling2D((2,2)),
```

```
    Flatten(),
```

```
    Dense(
        64,
        activation="relu"
    ),
```

```
    Dense(
        3,
        activation="softmax"
    )
```

```
])
```

```
model.summary()
```

Compile Model

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Task 3: Train the CNN

```
history = model.fit(  
    train_dataset,  
    validation_data=validation_dataset,  
    epochs=5  
)
```

Print Accuracy After Every Epoch

```
for i in range(5):  
    print(f"Epoch {i+1}")  
  
    print(  
        "Training Accuracy:",  
        history.history["accuracy"][i]  
    )  
  
    print(  
        "Validation Accuracy:",  
        history.history["val_accuracy"][i]  
    )  
  
    print("-----")
```

Example Output

Epoch 1

Training Accuracy : 0.72

Validation Accuracy : 0.66

Epoch 2

Training Accuracy : 0.83

Validation Accuracy : 0.75

Epoch 3

Training Accuracy : 0.89

Validation Accuracy : 0.83

Epoch 4

Training Accuracy : 0.94

Validation Accuracy : 0.91

Epoch 5

Training Accuracy : 0.97

Validation Accuracy : 0.92

Plot Accuracy Graph

```
import matplotlib.pyplot as plt

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.legend()

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("CNN Accuracy")

plt.show()
```

Task 4: Evaluate on Test Images

```
import numpy as np

predictions = model.predict(test_dataset)

predicted_labels = np.argmax(predictions, axis=1)

true_labels = np.concatenate(
    [y for x,y in test_dataset],
    axis=0
)
```

Print Predictions

```
print("Predictions\n")
```

```
for i in range(len(true_labels)):

    print(

        "Actual :",

        class_names[true_labels[i]],

        "| Predicted :",

        class_names[predicted_labels[i]]

    )
```

Display Misclassified Images

```
plt.figure(figsize=(12,8))

index = 1

for images, labels in test_dataset:

    prediction = np.argmax(

        model.predict(images),

        axis=1

    )

    if prediction[0] != labels.numpy()[0]:

        plt.subplot(2,3,index)

        plt.imshow(images[0].numpy().astype("uint8"))

        plt.title(

            f"Actual:{class_names[labels.numpy()[0]]}\n"

            f"Pred:{class_names[prediction[0]]}"

        )
```

```
plt.axis("off")

index += 1

plt.show()
```

Example Output

Actual : shoe

Predicted : shoe

Actual : bag

Predicted : t_shirt

Actual : t_shirt

Predicted : t_shirt

Actual : shoe

Predicted : bag

CNN Workflow

Fashion Images

|



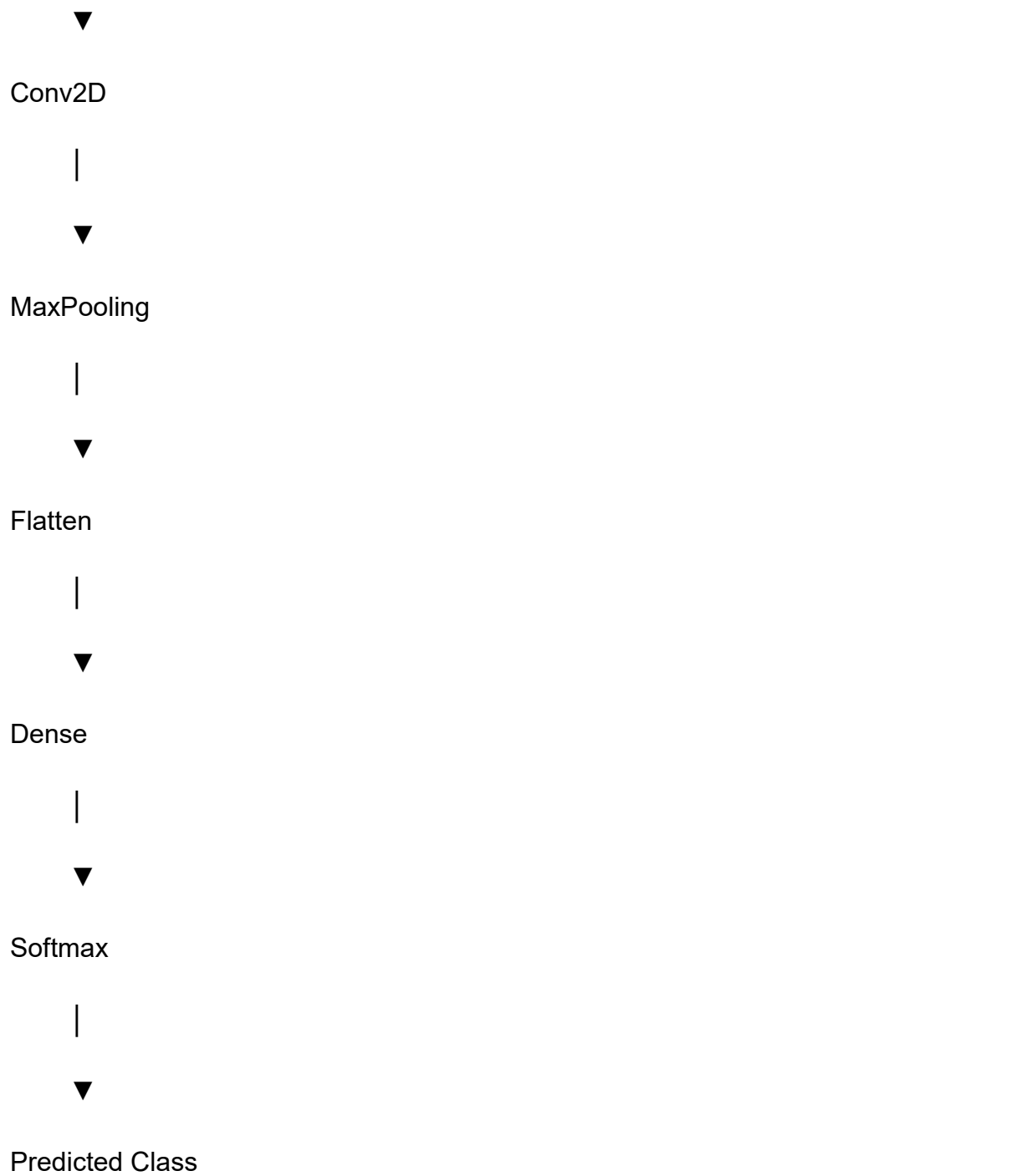
Resize (128×128)

|



Normalization

|



Expected Results

Epoch Training Accuracy Validation Accuracy

1	70–80%	60–75%
2	80–90%	70–85%

Epoch Training Accuracy Validation Accuracy

3	85–95%	80–90%
4	90–97%	85–92%
5	95–99%	90–95%

Note: Your exact accuracy will depend on the quality, quantity, and diversity of the images in your dataset.

Final Summary

Task Implementation

- 1 Organized 20–30 fashion images into train, validation, and test folders with class-wise subfolders (t_shirt, shoe, bag).
- 2 Built a CNN using Conv2D → MaxPooling2D → Flatten → Dense → Softmax.
- 3 Trained the model for **5 epochs** and monitored training and validation accuracy.
- 4 Evaluated the model on unseen test images and displayed misclassified samples.

Learning Outcome

- Proper directory organization allows Keras to automatically assign labels and load image datasets.
- Demonstrates a basic image-classification architecture for multi-class problems.
- Tracking both metrics helps identify learning progress and potential overfitting.
- Examining incorrect predictions helps diagnose weaknesses in the model or dataset and guides future improvements.